NuMagSANS Tutorial 1: HPC Installation

NuMagSANS Tutorials - Michael P. Adams

Version: July 7, 2026

Running the Tutorials

The executable tutorial scripts support two installation modes.

```
# Shared setup: local workstation or HPC shell
python -m venv .venv
source .venv/bin/activate

# Core install: data generation and NuMagSANS runs
pip install -e .

# Optional local plotting support
pip install -e ".[tutorials]"
```

- Use the core install for batch/HPC runs.
- Missing PyVista only disables diagnostic plots.
- Use the `tutorials` extra for local vector-field inspection.

Goal

Install NuMagSANS on an HPC system and verify that the CUDA backend and Python interface work in the same software environment.

- Start from an interactive GPU node.
- Load compiler, CUDA, CMake, Python, and optional HDF5 modules.
- Build the CUDA executable with CMake.
- Install the Python interface in a virtual environment.
- Run the shipped examples as installation checks.

Why HPC Installation Is Different

On a workstation, system tools are often globally installed. On an HPC system, the active software stack is usually defined by the job shell.

- The login node may not have a GPU.
- CUDA, compiler, CMake, Python, and HDF5 are usually loaded as modules.
- The module set must be consistent during build and run.
- Temporary directories and home quotas may be limited.
- Batch jobs need the same environment as the interactive test.

Portable Installation Pattern

The cluster-specific commands change, but the logic is stable.

```
# 1. Get a GPU shell or submit a GPU job.
# 2. Load the cluster modules.
# 3. Verify the tools.
# 4. Configure and build NuMagSANS.
# 5. Create a Python environment.
# 6. Run the examples as smoke tests.
```

The module names and partition names are the only parts that should be treated as site-specific.

Required Toolchain

Before building, the active shell should provide these commands.

```
nvidia-smi
nvcc --version
cmake --version
g++ --version
python --version
h5cc -showconfig | head -40
```

- `nvidia-smi` checks that the job sees an NVIDIA GPU.
- `nvcc` checks the CUDA toolkit.
- `cmake` configures the build.
- `g++` must support C++17.
- `h5cc` is needed only if the HDF5 backend is enabled.

Example: IRIS HPC

University of Luxembourg IRIS uses modules with site-specific names.

```
salloc -p interactive --gpus 1 --cpus-per-gpu 1 --mem 8G --time=60

module purge
module load lib/GDRCopy/2.4-GCCcore-13.2.0
module load lang/Python/3.11.5-GCCcore-13.2.0
module load system/CUDA/12.6.0
module load devel/CMake/3.27.6-GCCcore-13.2.0
module load data/HDF5
```

After loading modules, run the tool checks in the same shell.

Example: MPSD HPC

The MPSD Hamburg environment uses a different module family and GPU partition.

```
salloc -p gpu-interactive --gpus 1 --cpus-per-gpu 1 --mem 8G --time=60

module purge
mpsd-modules 25c native
module load gcc/13.2.0
module load python/3.11.7
module load cuda/12.6.2
module load cmake/3.27.9
module load hdf5/1.14.3
```

The structure is the same as on IRIS; only names and versions differ.

Build NuMagSANS

Clone and build in the same shell where the modules are loaded.

```
git clone https://github.com/AdamsMP92/NuMagSANS.git
cd NuMagSANS

cmake -S . -B build \
  -DCMAKE_CUDA_ARCHITECTURES=native \
  -DNUMAGSANS_ENABLE_HDF5=ON

cmake --build build -j
```

`CMAKE_CUDA_ARCHITECTURES=native` asks CMake to build for the GPU available in the allocated node.

Python Interface

Install the Python facade into a virtual environment created with the active cluster Python.

```
python -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -e .
python -c "from NuMagSANS import NuMagSANS; print('Installation successful')"
```

The editable install keeps the Python interface connected to the checked-out repository.

Smoke Tests

After building and installing, run the packaged examples.

```
cd examples/example1
python NuMagSANSrun.py

cd ../example2_csv
python NuMagSANSrun.py

cd ../example2_hdf5
python NuMagSANSrun.py
```

The CSV examples test the default output path. The HDF5 example additionally checks the optional HDF5 backend.

What Transfers To Other HPCs

For a new cluster, translate the two example installations into local choices.

- GPU partition name and resource request.
- CUDA module version and NVIDIA driver compatibility.
- C++17 compiler module.
- CMake module, ideally CMake 3.18 or newer.
- Python module used to create the virtual environment.
- HDF5 module if `NUMAGSANS_ENABLE_HDF5=ON` is used.
- Scratch or project path for builds and output data.

Common Pitfalls

Most installation failures are environment mismatches rather than NuMagSANS input errors.

- `nvidia-smi` fails: the shell is probably not on a GPU node.
- `nvcc` missing: CUDA toolkit module is not loaded.
- CMake finds the wrong compiler: load the compiler before configuring.
- HDF5 build fails: disable HDF5 or load a matching HDF5 module.
- No space left on device: set a private temporary directory.

```
mkdir -p $HOME/tmp
export TMPDIR=$HOME/tmp
```

Batch Job Reminder

The batch script should reproduce the tested interactive environment.

```
module purge
# load the same compiler, CUDA, CMake, Python, and HDF5 modules
source .venv/bin/activate

python tutorials/tutorial_4/run_numagsans_helical_sphere.py
```

Keep generated data and NuMagSANS output in project or scratch storage rather than in a small home directory.

Installation Checklist

- Allocate a GPU node.
- Load modules.
- Verify `nvidia-smi`, `nvcc`, `cmake`, `g++`, and `python`.
- Configure with the required output backends.
- Build with CMake.
- Install the Python package in a virtual environment.
- Run examples as smoke tests.
- Reuse the same module stack in production jobs.